

## AdvR 15 – Translating R Code

## Motivation

# Why Translate R Code?

Metaprogramming + lexical scoping + first-class environments → translate R code into **other languages**.

**dbplyr** powers this for SQL:

```
library(dbplyr)
con <- simulate_postgres()
translate_sql(x ^ 2, con = con)
#> <SQL> POWER(`x`, 2.0)
translate_sql(x < 5 & !is.na(x), con = con)
#> <SQL> `x` < 5.0 AND NOT((`x` IS NULL))
```

We'll build two simpler DSLs:

- 1 **HTML** — quasiquotation + data masks
- 2 **LaTeX** — tidy eval + AST walking

## HTML DSL

## Goal: R Code → HTML

Generate this HTML from R code that **mirrors its structure**:

```
with_html(body(  
  h1("A heading", id = "first"),  
  p("Some text &", b("some bold text.")),  
  img(src = "myimg.png", width = 100, height = 100)  
))
```

Three DSL properties:

- Nesting of function calls = nesting of tags
- Unnamed args → content; named args → attributes
- Special characters (&, <, >) auto-escaped

## Escaping with S3

An S3 class separates **raw text** (needs escaping) from **safe HTML**:

```
html <- function(x) structure(x, class = "advr_html")
escape <- function(x) UseMethod("escape")
escape.character <- function(x) {
  x <- gsub("&", "&amp;", x)
  x <- gsub("<", "&lt;", x)
  x <- gsub(">", "&gt;", x)
  html(x)
}
escape.advr_html <- function(x) x # already safe
```

```
c(escape("x > 1 & y < 2"),
  escape(html("<hr />"))) # not double-escaped
```

```
#> [1] "x &gt; 1 &amp; y &lt; 2" "<hr />"
```

## Separating Arguments: `dots_partition()`

Split ... into named (attributes) and unnamed (children):

```
dots_partition <- function(...) {  
  dots <- list2(...)  
  is_named <- if (is.null(names(dots)))  
    rep(FALSE, length(dots))  
  else names(dots) != ""  
  list(named = dots[is_named],  
       unnamed = dots[!is_named])  
}  
str(dots_partition(a = 1, 2, b = 3, 4))
```

```
#> List of 2  
#> $ named :List of 2  
#> ..$ a: num 1  
#> ..$ b: num 3  
#> $ unnamed:List of 2  
#> ... [output truncated]
```

## Building a Tag Function: `p()`

Unnamed args become content, named become HTML attributes:

```
p <- function(...) {  
  dots <- dots_partition(...)  
  attribs <- html_attributes(dots$named)  
  children <- map_chr(dots$unnamed, escape)  
  html(paste0("<p", attribs, ">",  
    paste(children, collapse = ""), "</p>"))  
}  
p("Some text", id = "myid")
```

```
#> <HTML> <p id='myid'>Some text</p>
```

`html_attributes()` converts a named list to HTML attribute strings (e.g., `id='myid'`). Implementation omitted — doesn't introduce new ideas.

## Tag Factory: `tag()`

Generalise `p()` to **any** tag with `rlang::new_function()` and `!!`:

```
tag <- function(tag) {  
  new_function(  
    exprs(... = ),  
    expr({  
      dots <- dots_partition(...)  
      attribs <- html_attributes(dots$named)  
      children <- map_chr(dots$unnamed, escape)  
      html(paste0(  
        !!paste0("<", tag), attribs, ">",  
        paste(children, collapse = ""),  
        !!paste0("</", tag, ">"))  
      }), caller_env()  
    })  
}  
  
p <- tag("p"); b <- tag("b"); i <- tag("i")  
p("Text. ", b(i("bold italic")), class = "x")
```

```
#> <HTML> <p class='x'>Text. <b><i>bold  
#> italic</i></b></p>
```

## Void Tags and `with_html()`

**Void tags** (<img>, <br>) have no children — `void_tag()` factory enforces attributes only.

Generate functions for all 100+ tags, store in a list, use as **data mask**:

```
html_tags <- c(
  tags %>% set_names() %>% map(tag),
  void_tags %>% set_names() %>% map(void_tag))
with_html <- function(code) {
  code <- enquo(code)
  eval_tidy(code, html_tags) # list as data mask!
}
with_html(body(
  h1("A heading", id = "first"),
  p("Some text &", b("some bold text.")),
  img(src = "myimg.png", width = 100,
      height = 100)))
```

```
#> <HTML> <body><h1 id='first'>A
#> heading</h1><p>Some text &amp;<b>some bold
#> text.</b></p><img src='myimg.png' width='100'
#> height='100' /></body>
```

## Exercises (HTML)

- 1 Predict the output. Will each call work or fail? Why?

```
greeting <- "Hello!"  
with_html(p(greeting))
```

```
p <- function() "p"  
address <- "123 anywhere street"  
with_html(p(address))
```

- 2 How would you modify `escape()` and `tag()` so that `<script>` tags escape `</script>` instead of `&`, `<`, `>`?

## Solutions (HTML)

- ① First call **works**: `p` comes from `html_tags` (data mask has precedence over `env`), `greeting` is found in the caller `env`. Second call **errors**: `address` is **also** an HTML tag in `html_tags`. Data mask has precedence, so `address` resolves to the tag function, not the string `"123 anywhere street"`.

```
greeting <- "Hello!"  
with_html(p(greeting))
```

```
#> <HTML> <p>Hello!</p>
```

```
p <- function() "p"  
address <- "123 anywhere street"  
with_html(p(address))
```

```
#> Error in `map_chr()`:
```

```
#> i In index: 1.
```

```
#> Caused by error in `UseMethod()`:
```

```
#> ! no applicable method for 'escape' applied to an object of class "function"
```

- ② Add a `script` argument to `escape()`: when `TRUE`, use `gsub("</script>", "<\\//script>", x)` instead of escaping `&`, `<`, `>`. Pass `!!script` into `tag()` so the generated function calls `escape(..., script = TRUE)`.

## LaTeX DSL

LaTeX math notation: `x * y`, `x1` (subscript), `\pi` ( $\pi$ ), `\frac{a}{b}` ( $\frac{a}{b}$ ), `\sqrt{x}` ( $\sqrt{x}$ ).

**Goal** — four translation rules:

| Input                           | Output                          | Rule                         |
|---------------------------------|---------------------------------|------------------------------|
| <code>\pi</code>                | <code>\pi</code>                | Known symbol                 |
| <code>x</code>                  | <code>x</code>                  | Unknown symbol (leave as-is) |
| <code>\sqrt{\frac{a}{b}}</code> | <code>\sqrt{\frac{a}{b}}</code> | Known function               |
| <code>f(a)</code>               | <code>\textrm{f}(a)</code>      | Unknown function             |

Unlike the HTML DSL, we need to **walk the AST** to handle unknown symbols and functions.

Capture the expression, evaluate in a **custom environment**:

```
latex <- function(x) structure(x, class = "advr_latex")
print.adv_latex <- function(x)
  cat("<LATEX> ", x, "\n", sep = "")

to_math <- function(x) {
  expr <- enexpr(x)
  out <- eval_bare(expr, latex_env(expr))
  latex(out)
}
```

Key difference from HTML DSL: the evaluation environment **varies** with the input expression (must handle unknown symbols/functions dynamically).

## Known Symbols: Greek Letters

Bind each Greek name to its LaTeX string in an environment:

```
greek <- c("alpha", "theta", "tau", "beta",  
  "pi", "gamma", "phi", "delta", "sigma",  
  "omega", "Gamma", "Lambda", "Sigma", "Psi")  
# ... 40 total  
greek_list <- set_names(paste0("\\", greek), greek)  
greek_env <- as_environment(greek_list)
```

When `eval_bare()` looks up `pi`, it finds `"\\pi"` in `greek_env`.

```
to_math(pi)
```

```
#> <LATEX> \pi
```

```
to_math(beta)
```

```
#> <LATEX> \beta
```

## Unknown Symbols: Walking the AST

Walk the AST to find all symbol names, create an environment mapping each to its string representation:

```
all_names_rec <- function(x) {  
  switch_expr(x,  
    constant = character(),  
    symbol = as.character(x),  
    call = flat_map_chr(as.list(x[-1]),  
      all_names))  
}  
all_names <- function(x) unique(all_names_rec(x))  
all_names(expr(x + y + f(a, b, 10)))
```

```
#> [1] "x" "y" "a" "b"
```

Chain: greek\_env (parent: symbol\_env) → Greek has priority over defaults. So `to_math(pi)` gives `\pi`, not `"pi"`.

## Function Factories: unary\_op and binary\_op

Helpers that manufacture LaTeX operator functions:

```
unary_op <- function(left, right) {  
  new_function(exprs(e1 = ),  
    expr(paste0(!!left, e1, !!right)),  
    caller_env())  
}  
binary_op <- function(sep) {  
  new_function(exprs(e1 = , e2 = ),  
    expr(paste0(e1, !!sep, e2)),  
    caller_env())  
}  
unary_op("\\sqrt{", "}")
```

```
#> function (e1)  
#> paste0("\\sqrt{", e1, "}")
```

These are **function factories**: each call returns a new function with the operator baked in via `!!`.

## Known Functions: `f_env`

Map R operators and functions to LaTeX equivalents:

```
f_env <- child_env(.parent = empty_env(),
  `+` = binary_op(" + "),
  `-` = binary_op(" - "),
  `*` = binary_op(" * "),
  `/` = binary_op(" / "),
  ``^` = binary_op("^"),
  `[` = binary_op("_"),          # subscript
  `( ` = unary_op("\\left( ", " \\right)"),
  sqrt = unary_op("\\sqrt{", "}"),
  sin = unary_op("\\sin(", ")"),
  frac = function(a, b)
    paste0("\\frac{", a, "}{" , b, "}"),
  hat = unary_op("\\hat{", "}"))
```

Lexical scoping lets us redefine `+`, `*`, `(`, etc. within this environment.

# Unknown Functions and the Complete DSL

Walk the AST for function names; wrap unknowns in `\textrm{}`:

```
unknown_op <- function(op) {  
  new_function(exprs(... = ), expr({  
    contents <- paste(..., collapse = ", ")  
    paste0("!!paste0("\\mathrm{", op, "}("),  
      contents, ")")  
  })))  
}
```

The **environment chain** (child  $\rightarrow$  parent):

greek\_env  $\rightarrow$  symbol\_env  $\rightarrow$  f\_env  $\rightarrow$  call\_env

```
to_math(sin(x + pi))
```

```
#> <LATEX> \sin(x + \pi)
```

```
to_math(sin(sin))
```

```
#> <LATEX> \sin(sin)
```

```
to_math(hat(beta))
```

```
#> <LATEX> \hat{\beta}
```

- ❶ What does `to_math(sin(sin))` return? Trace the environment lookup to explain why.
- ❷ The special characters `\`, `$`, `%` need escaping in LaTeX. Sketch how you'd add escaping using the same `S3` pattern as the HTML DSL.

- 1 Returns `\sin(sin)`. The function position `sin` matches `sin` in `f_env`  $\rightarrow$  `unary_op("\\sin(", ")")`. The **argument** `sin` matches the string `"sin"` in `symbol_env` (unknown symbols map to their own name). Functions take precedence because `f_env` is an ancestor of `symbol_env` — the argument is looked up in `symbol_env` first (closer to `greek_env`), where it finds the string.
- 2 Create an `advr_latex` S3 class with `escape_latex()` generic: `escape_latex.character()` prepends `\` to special chars, `escape_latex.adv_r_latex()` returns input unchanged (prevents double-escaping). Apply `escape_latex()` in `latex_env()` when creating the symbol environment and inside `unknown_op()`.